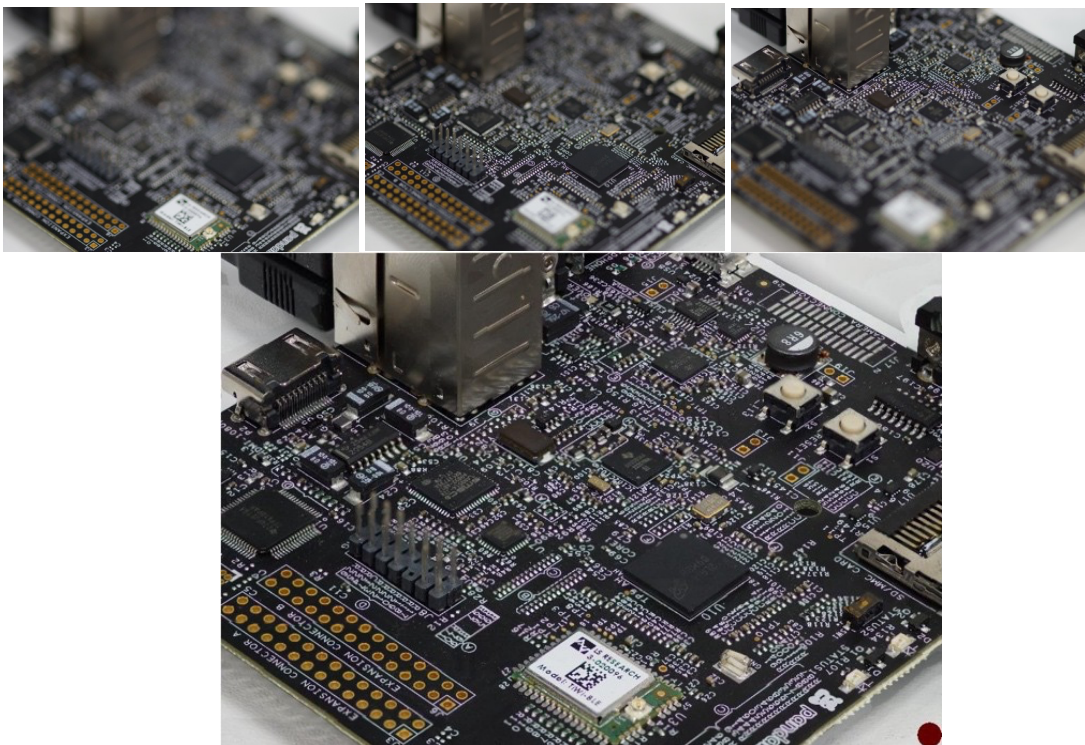


TP 3

1. **Re-focalisation d'une image à partir d'une séquence.** Le problème consiste à construire une image focalisée en tout point à partir d'une séquence de photos représentant la même scène, partiellement focalisée. Proposer et implémenter une solution.

Re-focusing of an image from a sequence. The problem is to build a focused image at any point from a sequence of photos representing the same scene, partially focused. Propose and implement a solution.



2. **Mosaïque d'images.** Réaliser une image panoramique à partir d'une séquence d'images (voir exemple ci-dessous). Assurer que la mosaïque se fait sans trou.

Images stitching. Take a panoramic image from a sequence of images (see example below). Ensure that the mosaic is done without holes.



3. **Redimensionner une image sans rompre le contenu.** Retirer ou insérer un chemin de pixel dans une image, de sorte que l'opération soit la plus imperceptible possible. Le principe consiste à trouver un chemin le moins coûteux pour aller d'un point A à un point B en définissant une fonction de coût (défini à partir du gradient par exemple et utiliser la programmation dynamique), puis à retirer le chemin de l'image. La figure ci-dessous montre le résultat après suppression de plusieurs de ces chemins (ex : Resise.png).

Context-Aware Image Resizing. Remove or insert a pixel path in an image, so that the operation is as imperceptible as possible. The principle consists in finding a least expensive way to go from point A to point B by defining a cost function (defined from the gradient for example and using dynamic programming), then removing the path from the picture. The figure below shows the result after deleting several of these paths (ex: Resise.png).



4. **Ajout de flou de profondeur de champs.** Cet exercice consiste à ajouter du flou de profondeur de champs sur une image qui n'en a pas. Il existe diverses façons d'y parvenir. La plus simple étant de faire manuellement une carte de profondeur de l'image : à chaque pixel de cette carte correspond une valeur, entre 0 et 255, représentant la profondeur dans la scène du pixel correspondant sur l'image originale. Ensuite, on applique sur chaque pixel de l'image originale un flou dont la puissance dépend de la profondeur associée au pixel, ainsi qu'à la profondeur de focus sélectionnée par l'utilisateur. Voici un exemple de ce qu'on peut obtenir :
(NB : Il existe d'autres approches plus performantes, comme celle de Kass)

*Interactive Depth of Field. This exercise consists in adding a blurring of depth of field on an image which does not have one. There are various ways to do this. The simplest being to manually make a depth map of the image: to each pixel of this map corresponds a value, between 0 and 255, representing the depth in the scene of the corresponding pixel on the original image. Then, a blur is applied to each pixel of the original image, the power of which depends on the depth associated with the pixel, as well as on the depth of focus selected by the user. Here is an example of what you can get:
(NB: There are other more effective approaches, like that of Kass)*



5. **Focaliser sur le centre d'intérêt d'une image sans trop déformer l'ensemble de l'image ni le contexte.** Mettre en évidence le centre de l'image afin de garder un souvenir similaire à celui enregistré par un touriste sur le site. Il s'agit de modifier le centre de l'image afin de le rendre plus percutant sans trop en modifier le contexte ni trop déformer l'ensemble de l'image. Une gaussienne 2D serait probablement une bonne solution (image de travail : tour-eiffel.jpg).

Focus on the center of interest of an image without distorting the overall image or context too much. Highlight the center of the image to retain a memory similar to that recorded by a tourist on the site. This involves modifying the center of the image to make it more striking, without altering the context or distorting the image too much. A 2D Gaussian would probably be a good solution (working image: tour-eiffel.jpg).



6. Ondelettes / Wavelets : débruitage & compression / denoising & compression

Rappel express : La DWT 2D sépare l'image en A_L (approximation) et détails H/V/D à plusieurs niveaux. Dénbruitage : seuillage des détails ; Compression : mise à zéro des petits coefficients.
Image d'exemple : `camera.png`. Placez-la dans le même répertoire que le notebook.

Quick Reminder: 2D DWT splits the image into A_L (approximation) and detail subbands H/V/D at multiple levels. Denoising: threshold details; Compression: zero-out small coefficients (sparsity).

Image d'exemple : image `camera.png`. Placez-la dans le même répertoire que le notebook.
Sample Image : We provide `camera.png`. Place it next to the notebook if needed.

https://drive.google.com/drive/folders/1b9Nes1W2NGX5bqnYKj7s5KD3Cs5fo7-L?usp=drive_link

Variantes rapides

- Changer wavelet : 'haar', 'db4', 'sym4', 'coif1'...
- Essayer level = 1 vs 3.
- Comparer **soft** vs **hard** thresholding (artefacts vs lissage).

Quick variants

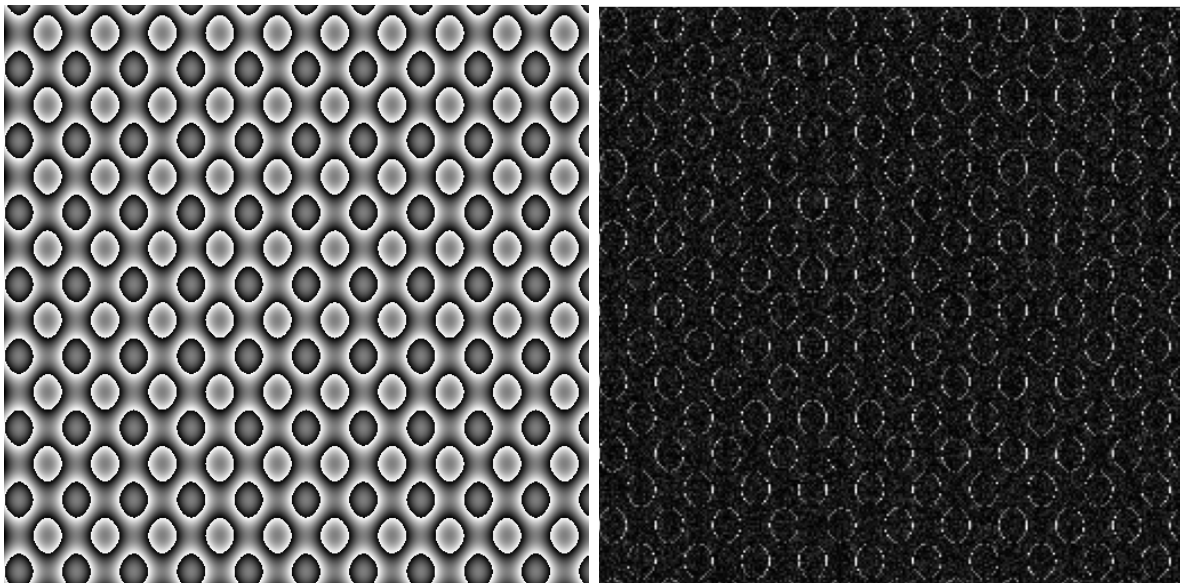
- Change the wavelet ('haar', 'db4', 'sym4', 'coif1').
- Vary the decomposition level ('level=1..3').
- Compare **soft** vs **hard** thresholding.

Questions

- Que capturent (H_1, V_1, D_1) (liens bords horiz/vert/diag) ?
- Comment évolue le débruitage quand on augmente/diminue le **seuil T** ?
- Jusqu'à quel **k%** de coefficients conservés la qualité reste-t-elle acceptable ?
- Pourquoi certaines ondelettes (ex. db4) produisent moins d'effets de "marches" que haar ?

Questions

- What do H1, V1, D1 capture?
- Effect of threshold T on denoising quality?
- Compression: minimal acceptable k%?
- Why does 'db4' reduce staircase artifacts vs 'haar'?



7. Filtres de Gabor / Gabor filters: Analyse de texture / Texture analysis

Rappel rapide : Un filtre de Gabor est une onde sinusoïdale modulée par une gaussienne :

$$g(x, y) = \exp(-(x'^2 + y'^2)/(2\sigma^2)) * \cos(2\pi x'/\lambda + \phi)$$

avec $x' = x \cos\theta + y \sin\theta$ et $y' = -x \sin\theta + y \cos\theta$.

Paramètres : λ (longueur d'onde), θ (orientation), ϕ (phase), σ (taille gaussienne), γ (rapport

d'aspect).

Quick Reminder: A Gabor filter is a sinusoidal wave modulated by a Gaussian envelope:

$$g(x, y) = \exp(-(x'^2 + y'^2)/(2\sigma^2)) * \cos(2\pi x'/\lambda + \phi)$$

with $x' = x \cos\theta + y \sin\theta$ and $y' = -x \sin\theta + y \cos\theta$.

Main parameters: λ (wavelength), θ (orientation), ϕ (phase), σ (Gaussian width), γ (aspect ratio).

Matériel fourni

- Image : lena_demo.png (intégrée et fournie).
- Notebook Jupyter : TP_Gabor_Lena_M1_FR.ipynb.

Provided Material

- Image: lena_demo.png (included).
- Jupyter Notebook: TP_Gabor_Lena_M1.ipynb.

Téléchargez le code et l'image depuis le google drive / Download the code / image on google drive :

https://drive.google.com/drive/folders/1gltJs8Xbhc7_wCuGR1xvX5cBsv_ge61S?usp=drive_link

et répondez aux questions suivantes, dans le code :

- Effet de la variation de θ (orientation) — quelles zones répondent le plus ?
- Effet de la variation de λ (fréquence spatiale) — quelles structures fines vs larges ?
- Intérêt de la normalisation du noyau (moyenne nulle, norme unité) avant convolution ?
- Bonus : esquisser une carte d'orientation (max sur θ , gestion de phase).

and reply to the next questions, in the code :

- How does the response change when θ varies? Identify oriented textures (edges, hair, fabric, etc.).
- What is the effect of λ (spatial frequency)? Which structures are better detected with small vs large values?
- Why normalize the kernel (zero mean, unit norm) before convolution?
- Bonus: how could you build an orientation map by aggregating responses (hint: max over θ , phase)?

